

Test

Giai đoạn

1. Bringup

Thành viên	Nhiệm vụ (Làm cái gì?)	Hướng dẫn thực hiện (Làm như thế nào?)	Kết quả đầu ra (Deliverables)	Tiêu chí đạt (Definition of Done)
MEMBER 2 (APB Agent)	Viết Kịch bản Test (Directed Sequence)	1. File: <code>tb/apb_agent/apb_generator.sv</code> 2. Sửa: Thay thế vòng lặp <code>randomize</code> bằng các gói tin cụ thể: - Ghi <code>0x03</code> vào <code>0x08</code> (Config 8-bit). - Ghi <code>0xAA</code> vào <code>0x00</code> (Data TX). - Ghi <code>0x01</code> vào <code>0x0C</code> (Start TX). 3. Logic: Phải tạo delay giữa các lệnh ghi để DUT kịp xử lý.	- File <code>apb_generator.sv</code> hoàn chỉnh. - Log in ra: <code>[APB_GEN] Configured UART</code>.	1. Mở sóng lên, thấy tín hiệu <code>pwdata</code> lần lượt là <code>0x03</code> → <code>0xAA</code> → <code>0x01</code> tại đúng các địa chỉ. 2. Không có báo lỗi <code>Setup/Hold time</code> vi phạm.
MEMBER 3 (UART Agent)	Code Logic Driver (Parity & Stop bits)	1. File: <code>tb/uart_agent/uart_driver.sv</code> 2. Sửa: Trong task <code>drive</code> : - Tính bit Parity (XOR các bit data) nếu <code>parity_en=1</code>. - Chèn bit Parity vào sau Data và trước Stop bit. - Xử lý <code>repeat(stop_bits)</code> để giữ mức cao đủ lâu.	- File <code>uart_driver.sv</code> có logic tính toán Parity. - Khả năng lái chân <code>cts_n</code> để chặn dòng.	1. Cấu hình Parity Lẻ → Soi sóng thấy bit Parity đúng là lẻ. 2. Cấu hình 2 Stop bit → Soi sóng thấy mức cao kéo dài gấp đôi bình thường.
MEMBER 4 (Scoreboard)	Xây dựng Shadow Register & Checker	1. File: <code>tb/env/scoreboard.sv</code> 2. Sửa: - Thêm biến <code>reg [31:0] shadow_cfg</code>. - Khi thấy APB ghi vào <code>0x08</code>, cập nhật <code>shadow_cfg</code>. - Khi check data: Nếu <code>shadow_cfg</code> là 5-bit, thực hiện <code>data & 0x1F</code> trước khi so sánh.	- File <code>scoreboard.sv</code> thông minh, biết lọc bit thừa. - Log báo <code>PASS / FAIL</code> chính xác.	1. Chạy test 5-bit mode: Gửi <code>0xFF</code> → Scoreboard báo <code>PASS</code> (vì nó hiểu chỉ cần 5 bit cuối). 2. Chạy test sai Parity → Scoreboard báo <code>FAIL</code>.

MEMBER 5 (Coverage)	Viết Coverage & Assertions	1. File: <code>tb/include/apb_if.sv</code> (Thêm <code>assert property</code>). 2. File: <code>tb/env/coverage.sv</code> (Thêm <code>covergroup</code>). 3. Logic: - Assert: <code>PENABLE</code> không được lên khi <code>PSEL</code> =0. - Cover: Đếm xem đã test đủ 4 loại data bit chưa?	- Báo cáo độ bao phủ (Coverage Report). - Các dòng Assertion error nếu vi phạm giao thức.	1. Coverage Report đạt 100% các bins (đã test đủ các trường hợp config). 2. Cố tình sửa sai driver → Assertion bắn lỗi đồ lôm ngay lập tức.
MEMBER 1 (Leader)	Tích hợp & Test Top-Level	1. File: <code>tb/test/test_tx.sv</code> , <code>test_rx.sv</code> . 2. Làm: Viết các bài test cụ thể gọi đến sequence của Member 2. 3. Review: Chạy <code>do run.do</code> và soi sóng của từng thành viên.	- Bộ testcase hoàn chỉnh (<code>run_all</code>). - File <code>run.do</code> chạy mượt mà.	1. Gõ <code>do run.do</code> → Kết quả cuối cùng là <code>Errors: 0</code> và Scoreboard báo <code>PASS</code> cho tất cả test case.

2. Kiểm tra chức năng

2.1 Kiểm tra tính năng cấu hình (Configuration)

Tính năng	Hành động của Testbench (Stimulus)	Kỳ vọng ở Thiết kế (Expected Behavior)	Dấu hiệu LỖI (Bug)
Data Bits (5-bit)	1. Config <code>0x8</code> = 5-bit mode. 2. Ghi data <code>0xFF</code> (11111111) vào TX.	- Trên sóng TX, chỉ được thấy 5 bit dữ liệu (5 bit thấp). - Scoreboard so sánh: <code>Actual == Expected & 0x1F</code> .	- Sóng TX vẫn bắn ra 8 bit. - Scoreboard báo <code>Mismatch</code> .
Parity Check (Odd)	1. Config Parity Enable = 1, Type = ODD. 2. Gửi data <code>0x03</code> (00000011 - có 2 bit 1).	- Bit Parity chèn vào phải là 1 (để tổng số bit 1 là 3 - số lẻ).	- Bit Parity trên sóng là 0 (thành chẵn). - Thiết kế tính sai công thức Parity.
Stop Bits (2-bit)	1. Config Stop bit = 2. 2. Gửi liên tiếp 2 gói tin.	- Giữa 2 gói tin phải có khoảng nghỉ (mức 1) dài bằng 2 chu kỳ bit.	- Chỉ nghỉ 1 chu kỳ bit rồi bắn gói tiếp theo ngay. - Thiết kế bỏ qua bit config này.

2.2 Kiểm tra tính năng luồng dữ liệu (Data path)

Tính năng	Hành động của Testbench (Stimulus)	Kỳ vọng ở Thiết kế (Expected Behavior)	Dấu hiệu LỖI (Bug)
-----------	------------------------------------	--	--------------------

TX Data Accuracy	Gửi một chuỗi data ngẫu nhiên: 0xAA , 0x55 , 0x12 , 0x34 .	- Chân TX bắn ra đúng thứ tự, đúng giá trị. - Scoreboard báo PASS 100%.	- Gửi 0xAA nhưng nhận được 0xAB (Sai bit). - Gửi 4 gói nhưng chỉ nhận được 3 gói (Mất gói).
RX Data Accuracy	UART Driver bắn data 0x7F vào chân RX của chip.	- Chip phải bật cờ RX_DONE trong thanh ghi Status. - CPU đọc địa chỉ 0x4 phải thấy 0x7F .	- Bắn xong mà cờ RX_DONE vẫn bằng 0. - Đọc ra thấy 0x00 hoặc rác.

2.3 Kiểm tra tính năng điều khiển luồng (Flow Control)

Tính năng	Hành động của Testbench (Stimulus)	Kỳ vọng ở Thiết kế (Expected Behavior)	Dấu hiệu LỖI (Bug)
CTS (Clear-to-Send)	- DUT đang bắn data hăng say. - UART Driver đột ngột kéo chân cts_n lên 1 (Báo bận).	- DUT phải dừng truyền ngay sau khi gửi xong byte hiện tại. - Chân TX giữ mức 1 (Idle).	- cts_n đã lên 1 mà DUT vẫn bắn data ầm ầm. ⇒ Lỗi vi phạm giao thức.
RTS (Request-to-Send)	- UART Driver bắn liên tiếp data vào RX. - CPU không chịu đọc data ra.	- Sau khi bộ đệm đầy (FIFO full), DUT phải kéo chân rts_n lên 1 để báo "Tao đầy rồi, đừng gửi nữa".	- Bộ đệm đầy nhưng rts_n vẫn bằng 0. ⇒ Dữ liệu mới đè lên dữ liệu cũ (Data Loss).

2.4 Bổ sung

Tính năng	Hành động (Stimulus)	Kỳ vọng (Expected)	Tại sao cần?
Error Injection (Tiêm lỗi)	Cấu hình Parity. Driver UART cố tình gửi 1 byte có sai bit Parity .	- DUT không được nhận data đó vào FIFO (hoặc nhận nhưng báo lỗi). - Cờ PARITY_ERROR (bit 2 reg Status) phải lên 1.	Kiểm tra xem mạch phát hiện lỗi có hoạt động không? Hay là "nhắm mắt nhận bừa".
FIFO Overrun (Tràn bộ nhớ)	Driver UART bắn liên tiếp 20 byte vào. CPU (APB Agent) cố tình không đọc .	- Sau byte thứ 16 (giả sử FIFO sâu 16), DUT phải kéo rts_n lên. - Nếu bắn tiếp byte 17, dữ liệu cũ không được bị mất, hoặc cờ OVERRUN (nếu có) phải báo.	Kiểm tra độ bền của mạch khi hệ thống bị quá tải.
Reset on-the-fly (Reset ngang xương)	Đang truyền dữ liệu (TX đang chạy, hoặc RX đang nhận), bất ngờ kéo rst_n xuống 0 rồi lên 1.	- Mạch phải về trạng thái mặc định ngay lập tức. - Chân TX phải lên 1	Đảm bảo mạch không bị "đơ" khi gặp sự cố điện.

	(Idle). Không được phép treo luôn.	
--	------------------------------------	--

3. Functional Coverage